EXPERIMENT 1

```
#include <stdio.h>
int main(){
int arr[50],n,i,j,min,temp;
printf("Enter no. of elements: ");
scanf("%d",&n);
printf("Enter Elements: ");
for(i=0;i<n;i++){
scanf("%d",&arr[i]);
}
for(int i=0;i<n-1;i++)
{
min = i;
for(int j = i+1; j<n;j++){
if(arr[j]<arr[min]){
min = j;
}}
temp=arr[i];
arr[i]=arr[min];
arr[min]=temp;
}
printf("Sorted Array:\n");
for(int i =0;i<n;i++){
printf(" %d ",arr[i]);
}
return 0;
}
```

OUTPUT:

```
Enter no. of elements: 5
Enter Elements: 1
4
9
8
7
Sorted Array:
14789
Process returned 0 (0x0)   execution time : 4.320 s
Press any key to continue.
```

```
Algorithm SELECTION SORT (A)
// A is an array of size n

for i = 0 to n - 1 do
     min = i
     for j = i+1 to n do
       if A[j] < A[min] do
          min = j;
       end if
    end for
    swap (A[min], A[i]);
end for
```

T(n)=O^2

S(n)=O(1)

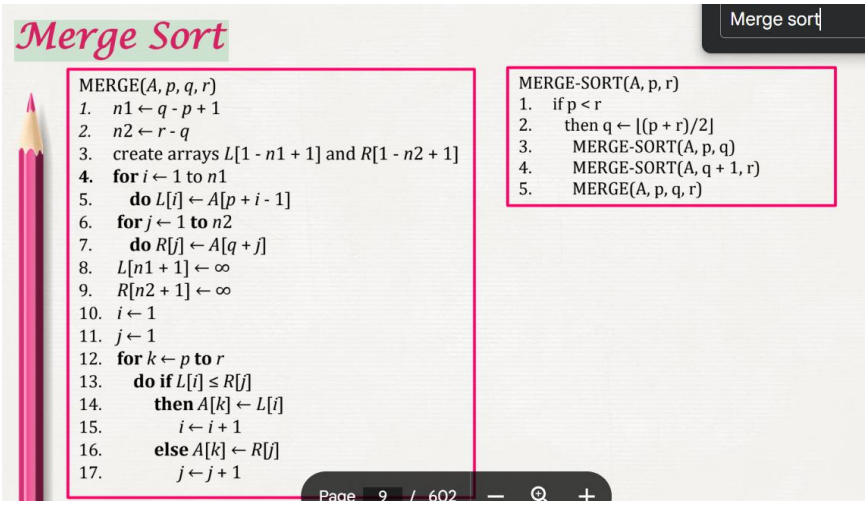
Approach: Iterative

PRACTICAL 2

```
#include <stdio.h>
void merge(int arr[], int left, int mid, int right)
{
   int i, j, k;
   int n1 = mid - left + 1;
   int n2 = right - mid;

   int L[n1], R[n2];

    for (i = 0; i < n1; i++)
      L[i] = arr[left + i];
   for (j = 0; j < n2; j++)
      R[j] = arr[mid + 1 + j];
   i = 0;
   j = 0;
   k = left;

   while (i < n1 && j < n2) {
      if (L[i] <= R[j]) {
         arr[k] = L[i];
         i++;
      } else {
         arr[k] = R[j];
         j++;
      }
      k++;
   }
   while (i < n1) {
      arr[k] = L[i];
      i++;
      k++;
   }

   while (j < n2) {
      arr[k] = R[j];
      j++;
      k++;
   }
}

void mergeSort(int arr[], int left, int right) {
   if (left < right) {
      int mid = left + (right - left) / 2;

      mergeSort(arr, left, mid);
      mergeSort(arr, mid + 1, right);
      merge(arr, left, mid, right);
   }
}

void printArray(int arr[], int n) {
   for (int i = 0; i < n; i++)
      printf("%d ", arr[i]);
   printf("\n");
}

int main() {
   int n;

   printf("Enter number of elements: ");
   scanf("%d", &n);

   int arr[n];

   printf("Enter %d elements:\n", n);
   for (int i = 0; i < n; i++)
      scanf("%d", &arr[i]);

   mergeSort(arr, 0, n - 1);

   printf("Sorted array:\n");
   printArray(arr, n);

   return 0;
}
```

OUTPUT:

```
Enter number of elements: 7
Enter 7 elements:
4
6
9
7
1
2
3
Sorted array:
1 2 3 4 6 7 9
```



Time Complexity: **O(N log N)**

Space Complexity: **O(N)**

Approach: The merge sort algorithm closely follows the divide-and-conquer paradigm. Intuitively, it operates as follows.

✓ Divide: Divide the n-element sequence to be sorted into two subsequences of n/2 elements each.

✓ Conquer: Sort the two subsequences recursively using merge sort.

✓ Combine: Merge the two sorted subsequences to produce the sorted answer

EXPERIMENT 3

```
#include <stdio.h>

void swap(int *a, int *b){
   int t = *a;
   *a = *b;
   *b = t;
}

int partition(int a[], int low, int high){
   int pivot = a[high];   // last element as pivot
   int i = low - 1;

   for(int j = low; j < high; j++){
      if(a[j] < pivot){
         swap(&a[++i], &a[j]);
      }
   }
   swap(&a[i+1], &a[high]);
   return i + 1;
}

void quickSort(int a[], int low, int high){
   while(low < high){
      int p = partition(a, low, high);

      // Optimize recursion (tail call optimization style)
      if(p - low < high - p){
         quickSort(a, low, p - 1);
         low = p + 1;
      } else {
         quickSort(a, p + 1, high);
         high = p - 1;
      }
   }
}

int main(){
   int n;
   printf("Enter no. of elements: ");
   scanf("%d", &n);

   int a[n];
   printf("Enter elements:\n");
   for(int i=0;i<n;i++)
      scanf("%d",&a[i]);

   quickSort(a,0,n-1);

   printf("Sorted array:\n");
   for(int i=0;i<n;i++)
      printf("%d ",a[i]);

   return 0;
}
```

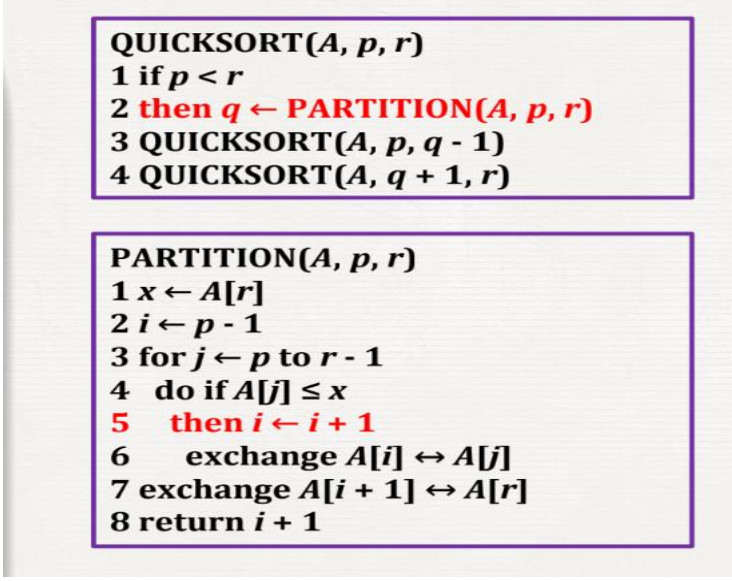
Approach:

Quicksort, like merge sort, is based on the divide-and-conquer paradigm

➢ Divide: Partition (rearrange) the array A[p, r] into two (possibly empty) subarrays A[p , q - 1] and A[q + 1 , r] such that each element of A[p , q - 1] is less than or equal to A[q], which is, in turn, less than or equal to each element of A[q + 1 , r]. Compute the index q as part of this partitioning procedure.

➢ Conquer: Sort the two subarrays A[p, q -1] and A[q +1, r] by recursive calls to quicksort.

➢ Combine: Since the subarrays are sorted in place, no work is needed to combine them: the entire array A[p r] is now sorted.



```
QUICKSORT(A, p, r)
1 if p < r
2 then q ← PARTITION(A, p, r)
3 QUICKSORT(A, p, q - 1)
4 QUICKSORT(A, q + 1, r)
```

```
PARTITION(A, p, r)
1 x ← A[r]
2 i ← p - 1
3 for j ← p to r - 1
4   do if A[j] ≤ x
5     then i ← i + 1
6       exchange A[i] ↔ A[j]
7 exchange A[i + 1] ↔ A[r]
8 return i + 1
```

Step5: Space Complexity

QUICK-SORT (A, p, r) : O(long n)

PARTITION(A, p, q, r): O(1)

So total space complexity is O(long n) + O(1) = O(Log n)

Time Complexity: Worst Case: $O(N^2)$

PRACTICAL 4

```
#include <stdio.h>
void swap(float *a, float *b){
   float temp = *a;
   *a = *b;
   *b = temp;
}
void swapInt(int *a, int *b){
   int temp = *a;
   *a = *b;
   *b = temp;
}
int partition(float ratio[], int profit[], int
weight[], int low, int high){
   float pivot = ratio[high];
   int i = low - 1;

   for(int j = low; j < high; j++){
      if(ratio[j] > pivot){
         i++;
         swap(&ratio[i], &ratio[j]);
         swapInt(&profit[i], &profit[j]);
         swapInt(&weight[i], &weight[j]);
      }
   }
   swap(&ratio[i+1], &ratio[high]);
   swapInt(&profit[i+1], &profit[high]);
   swapInt(&weight[i+1], &weight[high]);
   return i+1;
}
void quickSort(float ratio[], int profit[], int
weight[], int low, int high){
   if(low < high){
      int pi = partition(ratio, profit, weight, low,
high);
      quickSort(ratio, profit, weight, low, pi-1);
      quickSort(ratio, profit, weight, pi+1,
high);
   }
}
float fractionalKnapsack(int n, int profit[], int
weight[], int capacity){
   float ratio[n];

   for(int i = 0; i < n; i++){
      ratio[i] = (float)profit[i] / weight[i];
   }
quickSort(ratio, profit, weight, 0, n-1);
   float totalProfit = 0.0;

   for(int i = 0; i < n; i++){
      if(capacity >= weight[i]){
         totalProfit += profit[i];
         capacity -= weight[i];
      }
      else{
         totalProfit += profit[i]*((float)capacity
/ weight[i]);
         break;
      }
   }
      return totalProfit ;

   }
int main(){
   int n, capacity;
   printf("Enter number of items: ");
   scanf("%d",&n);
   int profit[n], weight[n];
   printf("Enter profit of items: ");
   for (int i = 0; i < n; i++){`
      scanf("%d", &profit[i]);
   }
   printf("Enter weight of items: ");
   for (int i = 0; i < n; i++){
      scanf("%d", &weight[i]);
   }
   printf("Enter Knapsack Capacity: ");
   scanf("%d", &capacity);

   float maxProfit = fractionalKnapsack(n,
profit, weight, capacity);

   printf("Maximum Profit is: %.2f\n",
maxProfit);
   return 0;
}
```

OUTPUT:

```
Enter number of items: 2
Enter profit of items: 25
30
Enter weight of items: 50
30
Enter Knapsack Capacity: 90
Maximum Profit is: 55.00
```

Step1: Approach

Fractional Kanpsack uses greedy approach. For each decision point in the algorithm,

the choice that seems best at the moment is chosen.

```
Fractional Knapsack (X, W,  V, M)
1.    for i <- 1 n
2.        calculate cost[i] <- V[i] / W[i]
3.    Sort-Descending (cost)
4.    S ← Ø
5.    SW ← 0
6.    SP ← 0
7.    i ← 1
8.    while (i <= n)
9.          if  (SW+W[i] )<= M
10.               S ← S ∪ X[i]
11.               SW ← SW+ W[i]
12.               SP ← SP+ V[i]
13.         else
14.               frac ← (M-SW)/W[i]
15.               S ← S ∪ X[i]
16.               SW ← SW+ W[i] * frac
17.               SP ← SP+ V[i] * frac
18.               goto 20
19.   i ← i + 1
20.   End
```

Step4: Time Complexity

Time complexity is O (n log n)

Step5: Space Complexity

Space Complexity is: O(n)

EXPERIMENT 5

```
#include <stdio.h>
int u[20],v[20],w[20],parent[20];
void swap(int i,int j)
{
int t;
t=w[i]; w[i]=w[j]; w[j]=t;
t=u[i]; u[i]=u[j]; u[j]=t;
t=v[i]; v[i]=v[j]; v[j]=t;
}
int partition(int low,int high)
{
int pivot=w[high];
int i=low-1,j;
for(j=low;j<high;j++)
{
if(w[j] < pivot)
{
i++;
swap(i,j);
}
}
swap(i+1,high);
return i+1;
}
void quicksort(int low,int high)
{
if(low<high)
{
int p=partition(low,high);
quicksort(low,p-1);
quicksort(p+1,high);
}
}
int find(int i)
{
while(parent[i])
i=parent[i];
return i;
}
int unionSet(int i,int j)
{
if(i!=j)
{
parent[j]=i;
return 1;
}
return 0;
}
int main()
{
int n,e,i,a,b,ne=0,mincost=0;
printf("Enter number of vertices and edges:");
scanf("%d%d",&n,&e);
for(i=0;i<e;i++)
{
printf("Enter edge(u v) and weight: ");
scanf("%d%d%d",&u[i],&v[i],&w[i]);
}
quicksort(0,e-1);
for(i=0;i<e && ne<n-1;i++)
{
a=find(u[i]);
b=find(v[i]);
if(unionSet(a,b))
{
printf("Edge (%d
%d)cost=%d\n",u[i],v[i],w[i]);
mincost+=w[i];
ne++;
}
}
printf("Minimum cost=%d",mincost);
}
```

OUTPUT:

```
Enter number of vertices and edges:3 4
Enter edge(u v) and weight: 4 6 8
Enter edge(u v) and weight: 2 3 7
Enter edge(u v) and weight: 1 7 8
Enter edge(u v) and weight: 4 7 1
Edge (4 7)cost=1
Edge (2 3)cost=7
Minimum cost=8
```

Step1: Approach

Kruskal's Algorithm uses greedy approach. For each decision point in the algorithm,

the choice that seems best at the moment is chosen.

```
MST-KRUSKAL(G, w)
1.  A ← Ø
2.  for each vertex v ∈ V[G]
3.      do MAKE-SET(v)
4.  sort the edges of E into nondecreasing order by weight w
5.  for each edge (u, v) E, taken in nondecreasing order by weight
6.      do if FIND-SET(u) ≠ FIND-SET(v)
7.          then A ← A {(u, v)}
8.               UNION(u, v)
9.  return A
```

Step4: Time Complexity

Time complexity is O (E log E) / O (E log V)

Step5: Space Complexity

Space Complexity is: O (V + E)